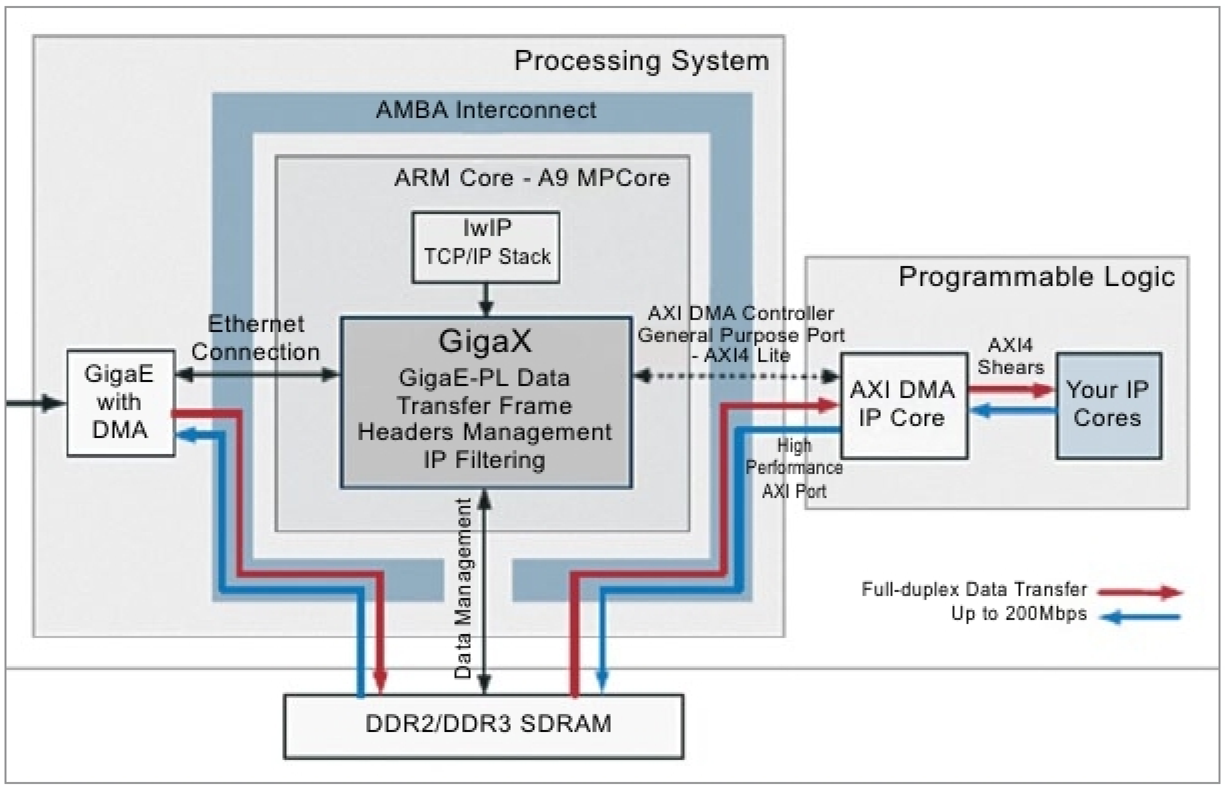

Fig. 3: Zynq GigaX functionality and interfaces

marked as final, when this input is activated, the corresponding task is activated irrespective of the status of other associated inputs.

All functionalities associated with a task are provided in the task. This forms an important part of the API. Processing by each task is performed by overloading its execute method.

Task input. This is an essential part of each task. Data to be processed are passed on to a task as inputs. Inputs are associated with channels. The relation is 1:N. Here, one input is associated with only one channel, and each channel can be associated with N inputs.

Functionalities are provided in the task input. It is possible to control activation of an input by specifying a threshold for the number of activations. There is also a flag indicating an input as final, which, when available, activates the task irrespective of the status of other inputs. This is included as a time-out mechanism to avoid a task from waiting for more than a predecided time.

Task channel. This is the base for data container implementation. It stores and distributes the data for multiple tasks. Each channel can be associated with more than two inputs. A push method is provided, which notifies all associated task inputs that a new set of data is available on the channel.

In a practical scenario, many sensors associated with a satellite are designed to send data at a specific time. To serve such recurring process requests, a task event is provided in the API.

Task events are not tasks. These are an extension to the task channel. By default, these are designed to execute a push method to push data on to the channel at a specified time. Two types of events are supported by the framework, one which executes on a periodic basis and other which is executed after a specified time lapse after the call to reset that channel is made.

Bare-metal porting of the tasking framework on a board means programming without various layers of abstraction or, as some would call it, programming without an OS.

The application is the only software running on a processor in bare-metal implementations. This is because an OS uses more resources like RAM and flash for a simple application. These resources are available in limited quantity when working with embedded systems.

The application processor unit (APU) used is ARM Cortex A9 processor, which is based on ARM v7 architecture.

Cortex-A9 MPCore. One to four Cortex-A9 processors in a cluster and a snoop control unit (SCU) can be used to ensure coherency within the cluster. A set of private memory-